Cas pratique : développement et suivi du cycle de vie d'une application

- VéloVille -

M2I

- RNCP 39103 –

Épreuve 02

Thomas Lerond

Sommaire

1. Contexte et périmètre
2. Démonstration
3. Choix techniques et qualité
4. Limites et axes d'amélioration

1. Contexte et périmètre

- Besoins
- Architecture retenue
- Ce qui est livré et ce qui ne l'est pas.

Besoins

VéloVille HdF

- *120 stations*
- *1800 vélos*
- *9000 trajets/jour*

- *6 Agents de rééquilibrage*
- *4 Techniciens de maintenance*
- *1 Coordinatrice*
- *2 développeurs*

Existant :

- Les exports ne sont pas fiabilisés
- Aucun référentiel commun
- Les indicateurs mensuels sont produits à la main
- Les fichiers circulent par courriel, sans contrôle d'accès ni traçabilité.

Solution :

- **Exports fiabilisés**
- **Référentiel commun**
- **Indicateurs automatisés**
- **Fichiers sécurisés**

Architecture retenue

- **Monolithe modulaire**

  Pourquoi ?

  → Petite équipe, projet court terme

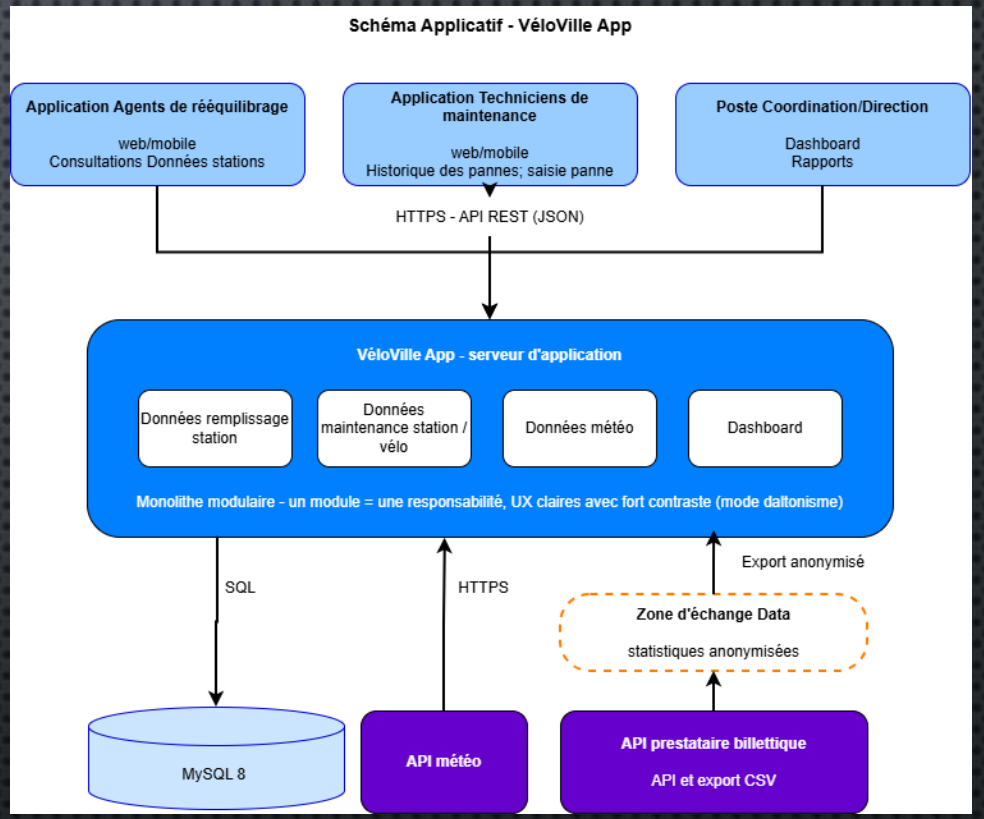


Schéma applicatif issu de ma proposition de CdC

Ce qui est livré / Ce qui ne l'est pas

Solution contenant :

- Script de fiabilisation et d'importation des données
- Gestion de BDD par Postgre SQL
- API backend contenant les fonctionnalités suivantes :
  - Authentification
  - Visualisation de la liste des stations
  - Recherche d'une station particulière
  - Visualisation des trajets
  - Visualisation des différents incidents
  - Mise à jour du statut d'un incident
  - Visualisation des indicateurs :
    - Activité par jour
    - Stations actives
    - Nombre d'incidents par statut
- Frontend Streamlit permettant une navigation entre les fonctionnalités de l'API avec restitution tabulaire

Hors périmètre :

- Création d'un nouveau compte
- Suivi météo

2. Démonstration

- Audit et import des données (journal des rejets, idempotence)
- Connexion avec les deux comptes et différence de droits (un 403 montré)
- Les quatre écrans du front
- Un indicateur rapproché d'une requête SQL de contrôle
- Exécution des tests et affichage de la couverture
- Exécution de la sauvegarde

Audit et import des données

- Nettoyage des csv
  - Vérification des règles d'écriture des dates et heures
  - Normalisation des entrées écrites
  - Suppression des lignes physiquement impossible (restitution d'un vélo avant son emprunt)
- Double sortie : csv et importation dans la BDD
- Rédaction automatisée d'un journal de fiabilisation

Bilan de la dernière fiabilisation (2026-10-14 16:46:33)

1. Synthèse par entité

Entité	Lignes lues	Valides (Chargées)	Avertissements (Modifiées)	Rejets (Erreurs)
Stations	120	120	0	0
Trajets	2030	2009	6	21
Incidents	180	177	177	3

2. Journal des rejets (Lignes exclues)

Trajets

- **Ligne 31** : Station d'arrivée inconnue ou invalide : 'S999'
- **Ligne 35** : Station d'arrivée inconnue ou invalide : 'S999'
- **Ligne 155** : Format d'horodatage invalide : '24/08/2026 11:01'
- **Ligne 238** : Station d'arrivée inconnue ou invalide : 'S999'
- **Ligne 384** : Format d'horodatage invalide : '25/08/2026 17:08'

Différence de droits

Gestion des Incidents

Liste des incidents

Filtrer par commune

Taille de page

20 − +

Erreur API (403) : Acces reserve au role exploitant

VéloVille

L'application de gestion des vélos en libre-service de la ville — ver.1.0.1

Connecté en tant que **exploitant01** (Rôle : `exploitant`)

Se déconnecter

Gestion des Incidents

Liste des incidents

Filtrer par commune

Taille de page

20 − +

Page

1 − +

Total : 177 incident(s) (Page 1/9)

incident_id	cible	velo_id	station_id	statut	date_signalement	date_resolution
I2026-0001	velo	V0745	None	resolu	2026-08-05T15:00:00	2026-10-15T08:
I2026-0002	station	None	S075	ouvert	2026-08-26T20:00:00	None
I2026-0003	velo	V0447	None	resolu	2026-07-04T13:00:00	2026-07-07T14:
I2026-0004	velo	V1476	None	resolu	2026-07-08T10:00:00	2026-07-16T19:
I2026-0005	station	None	S065	ouvert	2026-08-09T09:00:00	None
I2026-0006	station	None	S109	resolu	2026-07-15T11:00:00	2026-07-24T22:
I2026-0007	station	None	S045	ouvert	2026-08-05T18:00:00	None
I2026-0008	velo	V0906	None	en_cours	2026-08-10T16:00:00	None
I2026-0009	velo	V0528	None	resolu	2026-08-25T14:00:00	2026-08-28T06:
I2026-0010	velo	V1381	None	ouvert	2026-07-13T09:00:00	None

Le Frontend

VéloVille

L'application de gestion des vélos en libre-service de la ville — ver.1.0.1

Connecté en tant que **exploitant01** (Rôle : `exploitant`)

Se déconnecter

Gestion des Stations

Filtrer par commune

Taille de page 20

Page 1

Total : 120 station(s) trouvée(s) (Page 1/6)

station_id	nom	commune	capacite	latitude	longitude
S001	Lille - Gare	Lille	15	50.5993	3.2195
S002	Lille - Mairie	Lille	30	50.5823	3.0288
S003	Lille - Place de la Republique	Lille	35	50.6847	3.2436
S004	Lille - Grand Place	Lille	20	50.5508	3.0832
S005	Lille - Universite	Lille	30	50.6744	3.2206
S006	Lille - Stade	Lille	30	50.7035	3.0285
S007	Lille - Piscine	Lille	25	50.5844	3.1548
S008	Lille - Theatre	Lille	40	50.6064	3.0611
S009	Lille - Marche	Lille	40	50.6368	2.9546
S010	Lille - Parc	Lille	25	50.7205	3.0984

Gestion des Incidents

Liste des incidents

Filtrer par commune

Taille de page 20

Page 1

Total : 177 incident(s) (Page 1/9)

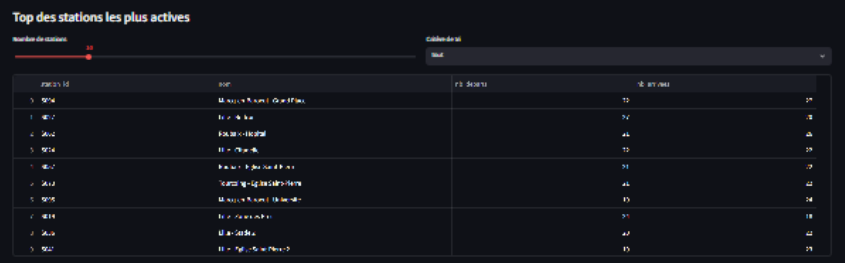
incident_id	cible	velo_id	station_id	statut	date_signalement	date_resolution
I2026-0001	velo	V0745	None	resolu	2026-08-05T15:00:00	2026-10-15T08:
I2026-0002	station	None	S075	ouvert	2026-08-26T20:00:00	None
I2026-0003	velo	V0447	None	resolu	2026-07-04T13:00:00	2026-07-07T14:
I2026-0004	velo	V1476	None	resolu	2026-07-08T10:00:00	2026-07-16T19:
I2026-0005	station	None	S065	ouvert	2026-08-09T09:00:00	None
I2026-0006	station	None	S109	resolu	2026-07-15T11:00:00	2026-07-24T22:
I2026-0007	station	None	S045	ouvert	2026-08-05T18:00:00	None
I2026-0008	velo	V0906	None	en_cours	2026-08-10T16:00:00	None
I2026-0009	velo	V0528	None	resolu	2026-08-25T14:00:00	2026-08-28T06:
I2026-0010	velo	V1381	None	ouvert	2026-07-13T09:00:00	None

Indicateurs Métiers

Activité des trajets par jour

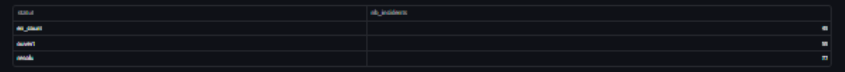


Top des stations les plus actives



Répartition et résolution des incidents

149.8 heures

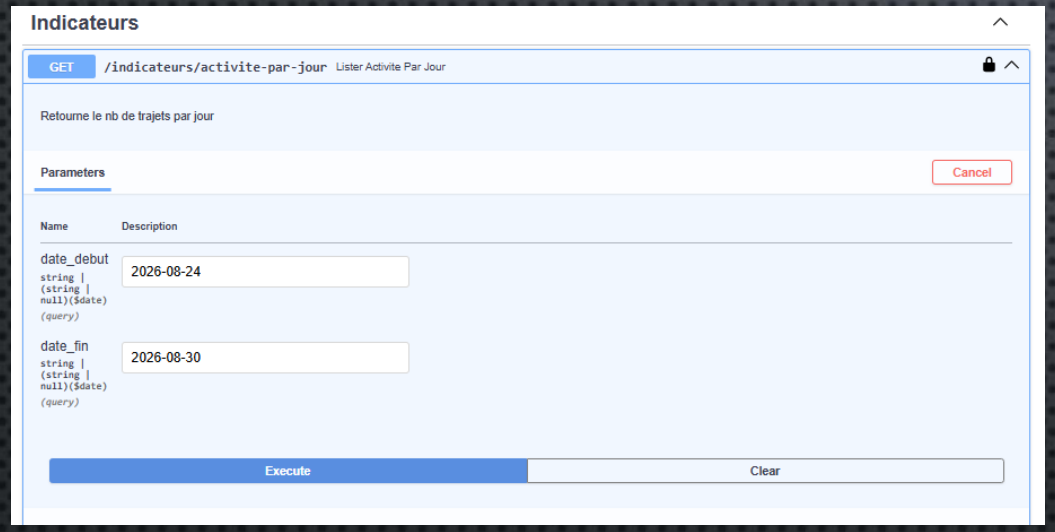


Je remarque en rédigeant ce petit bug d'affichage
Absent de la BDD

date_signalement
timestamp without time zone

2026-08-05 15:00:00

Un indicateur rapproché d'une requête SQL de contrôle





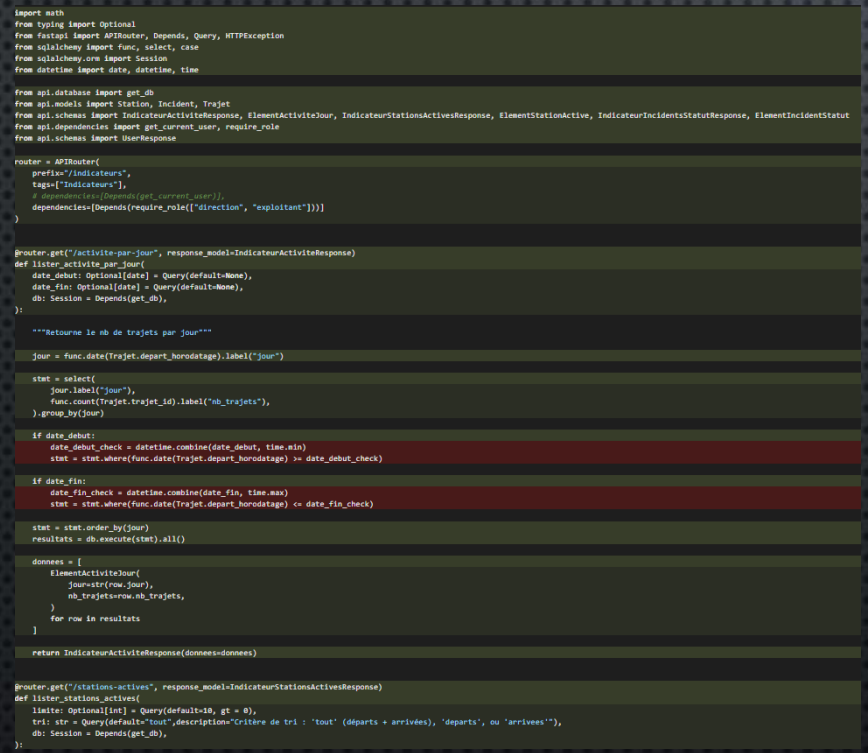
Issu du cahier de recettes :
REC-09 :
Cohérence de l'indicateur « activité par jour » avec la base

```sql
SELECT depart_horodatage::date AS jour,
    count(*) AS nb_trajets FROM trajets
        WHERE depart_horodatage::date BETWEEN '2026-08-24' AND '2026-08-30'
        GROUP BY jour
        ORDER BY jour;
```

	jour date	nb_trajets bigint
1	2026-08-24	329
2	2026-08-25	310
3	2026-08-26	306
4	2026-08-27	300
5	2026-08-28	315
6	2026-08-29	218
7	2026-08-30	220

Exécution des tests et affichage de la couverture

file:///C:/Users/Administrateur/Desktop/Cours/Evaluation%20Bloc%202/epreuve-e2-veloville/htmlcov/index.html



```python
import math
from typing import Optional
from fastapi import APIRouter, Depends, Query, HTTPException
from sqlalchemy import func, select, case
from sqlalchemy.orm import Session
from datetime import date, datetime, time

from api.database import get_db
from api.models import Station, Incident, Trajet
from api.schemas import IndicateurActiviteResponse, ElementActiviteJour, IndicateurStationsActivesResponse, ElementStationActive, IndicateurIncidentsStatutResponse, ElementIncidentStatut
from api.dependencies import get_current_user, require_role
from api.schemas import UserResponse

router = APIRouter(
    prefix="/indicateurs",
    tags=["Indicateurs"],
    # dependencies=[Depends(get_current_user)],
    dependencies=[Depends(require_role(["direction", "exploitant"]))]
)


@router.get("/activite-par-jour", response_model=IndicateurActiviteResponse)
def lister_activite_par_jour(
    date_debut: Optional[date] = Query(default=None),
    date_fin: Optional[date] = Query(default=None),
    db: Session = Depends(get_db),
):
    """Retourne le nb de trajets par jour"""

    jour = func.date(Trajet.depart_horodatage).label("jour")

    stmt = select(
        jour.label("jour"),
        func.count(Trajet.trajet_id).label("nb_trajets"),
    ).group_by(jour)

    if date_debut:
        date_debut_check = datetime.combine(date_debut, time.min)
        stmt = stmt.where(func.date(Trajet.depart_horodatage) >= date_debut_check)

    if date_fin:
        date_fin_check = datetime.combine(date_fin, time.max)
        stmt = stmt.where(func.date(Trajet.depart_horodatage) <= date_fin_check)

    stmt = stmt.order_by(jour)
    resultats = db.execute(stmt).all()

    donnees = [
        ElementActiviteJour(
            jour=str(row.jour),
            nb_trajets=row.nb_trajets,
        )
        for row in resultats
    ]

    return IndicateurActiviteResponse(donnees=donnees)


@router.get("/stations-actives", response_model=IndicateurStationsActivesResponse)
def lister_stations_actives(
    limite: Optional[int] = Query(default=10, gt = 0),
    tri: str = Query(default="tout",description="Critère de tri : 'tout' (départs + arrivées), 'departs', ou 'arrivees'"),
    db: Session = Depends(get_db),
):
```

VeloVille --- Cas Pratique --- Thomas Lerond

Coverage report: 83%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-15 14:51 +0200

File ▲	statements	missing	excluded	coverage
ressources\squelette-git\api__init__.py	0	0	0	100%
ressources\squelette-git\api\config.py	7	0	0	100%
ressources\squelette-git\api\database.py	13	4	0	69%
ressources\squelette-git\api\dependencies.py	27	6	0	78%
ressources\squelette-git\api\main.py	14	2	0	86%
ressources\squelette-git\api\models.py	40	0	0	100%
ressources\squelette-git\api\routers__init__.py	0	0	0	100%
ressources\squelette-git\api\routers\auth.py	26	9	0	65%
ressources\squelette-git\api\routers\incidents.py	40	3	0	92%
ressources\squelette-git\api\routers\indicateurs.py	51	6	0	88%
ressources\squelette-git\api\routers\stations.py	29	6	0	79%
ressources\squelette-git\api\routers\trajets.py	32	9	0	72%
ressources\squelette-git\api\schemas.py	36	0	0	100%
ressources\squelette-git\api\securite.py	17	3	0	82%
ressources\squelette-git\api\services__init__.py	0	0	0	100%
ressources\squelette-git\api\services\fiabilisation.py	125	29	0	77%
ressources\squelette-git\api\services\indicateurs.py	0	0	0	100%
Total	457	77	0	83%

coverage.py v7.16.2, created at 2026-10-15 14:51 +0200

3. Choix techniques et qualité

3. Résultats de la campagne

10 cas prévus, 10 cas éxécutés :

- 6 OK
- 4 KO
- 1 NT

- Structure du code

  Pour le code, 3 sous-dossiers principaux :

  - api → backend
  - front → frontend
  - scripts → script d'importation des csv
- Tests

  Un dossier de tests à part, rapport automtisé en html
- Sécurité

  2 profils différents avec des accès différents

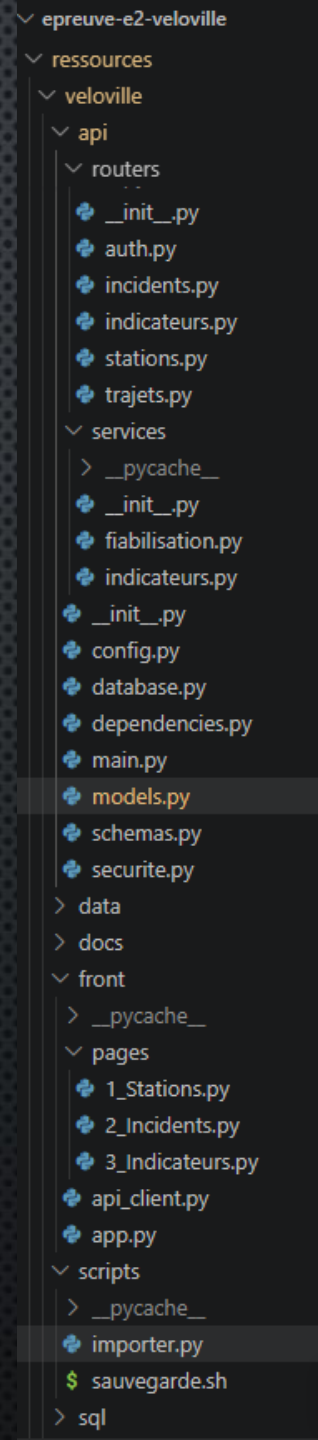
- Écoconception

  frontend qui répond sur demande, pas en continu

  pagination à base de requêtes SQL pour ne pas charger la BDD entière
- Recette

  bilan des recettes sur la ver.1.0.0

  corrections en cours sur la ver 1.0.1



4. Limites et axes d'amélioration

- 5 anomalies détectées en cours de recettage
  - 2 ont été corrigées en v.1.0.1
  - 3 sont en cours

- Pour la v.2 :

  Nouvelles fonctionnalités prévues :
    - Création d'un nouveau compte
    - Suivi météo